AI Itinerary Generator:

AI-Based Intelligent Multi-Day Trip Planning Platform for Sri Lanka Tourism

Project ID – R26-IT-152

Project Proposal Report

Wanniarachchi P.M.R — IT22225092

Bachelor of Science (Hons) Degree in Information Technology
Specializing in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology (SLIIT)
Sri Lanka

March 2026

TABLE OF CONTENTS

LIST OF FIGURES iii
LIST OF TABLES iii
ABSTRACT iv
ACKNOWLEDGEMENT v
LIST OF ABBREVIATIONS vi
LIST OF APPENDICES vi
1 INTRODUCTION 1
1.1 Background and Context 1
1.2 Literature Review 3
1.3 Research Gap 4
2 OBJECTIVES 6
2.1 Main Objective 6
2.2 Specific Objectives 6
3 METHODOLOGY 7
3.1 Research Approach 7
3.2 Random Forest Model Stack 7
3.3 Training and Validation 8
3.4 Deterministic Rules and Heuristics 8
3.5 Tools and Technologies 9
3.6 Data Collection and Ethical Considerations 9
4 HIGH-LEVEL ARCHITECTURE 10
4.1 Three-Layer Architecture 10
4.2 Data Flow 10
4.3 Scalability and Reliability 11
5 USER REQUIREMENTS 12
6 COMMERCIALIZATION PLANS 14
7 GANTT CHART 15
8 SYSTEM DIAGRAM 16
9 CONCLUSION 17
10 BUDGET AND BUDGET JUSTIFICATION 18
11 WORK BREAKDOWN STRUCTURE 19
12 REFERENCES 20
13 APPENDICES 21

LIST OF FIGURES

Figure 1: Three-Layer System Architecture 10
Figure 2: End-to-End Data Flow of Itinerary Generation 11
Figure 3: Gantt Chart — March to October 2026 Development Timeline 15
Figure 4: Random Forest Training and Deployment Pipeline 16

LIST OF TABLES

Table 1: Comparison of Existing Systems and Their Limitations 1
Table 2: Mapping of the Proposed Solution to Implementation Modules 2
Table 3: Summary of Literature Review 3
Table 4: Specific Objectives and Deliverables 6
Table 5: Random Forest Model Stack (rf_bundle.joblib) 7
Table 6: Technology Selection and Justification 9
Table 7: Functional Requirements 12
Table 8: Non-Functional Requirements 13
Table 9: Budget Breakdown and Justification 18
Table 10: Work Breakdown Structure 19

ABSTRACT

In 2025, Sri Lanka recorded 2,360,000 international arrivals, with a national target of three million by the end of 2026 [1]. Despite this growth, independent travellers still plan multi-day routes manually. A tourist typically wants to describe the desired experience in free text, provide airport arrival and departure dates and times, a nightly budget, a comfort level, and a meal plan (BB/HB/FB), and receive a feasible multi-day plan in which attractions respect their official visit start and end times, hotels are drawn from the registered accommodation list, and no two visits overlap on the same day. No existing tool performs this end to end on Sri Lankan catalogue data.

This research proposes the AI Itinerary Generator, a web module built with a React wizard, a Flask REST backend, PostgreSQL, and an offline-trained Random Forest bundle (`rf_bundle.joblib`). The implementation is grounded in two Excel catalogues: `attractions.xlsx` (All Data and Destination Alias sheets — attraction name, destination, category, moods, visit start/end/duration, activities, details, rating) and `accomadation.xlsx` (hotel name, category, rating, local authority, address, contact, and BB/HB/FB minimum/maximum prices in LKR). `build_training_data.py` copies these catalogues into the RF package and builds `RF_training_data.xlsx` with eight sheets (Trips, Train_Attractions, Train_Hotels, Train_Visit_Times, Train_PPD, Train_Destinations, Train_Itinerary_Samples, Geo). `train_models.py` then trains the model stack and saves it, with joblib compression level 3, as `models/rf_bundle.joblib`. At runtime, `generate_itinerary.generate()` and the backend service `rf_itinerary_service.py` only perform inference; no model is trained during an HTTP request.

The system produces attraction suggestions, a hotel pick per destination (with the full hotel catalogue scored), and day-by-day rows: arrival and hotel check-in on day one, timed attraction visits on middle days only, and free time/shopping followed by departure on the last day. Evaluation uses metrics printed by the training script — micro-F1 for need (mood/category) detection and AUC for attraction and hotel ranking, with GroupKFold cross-validation grouped by `trip_id` — and the whole pipeline is reproducible via `RETRAIN_ALL_RF.bat` / `scripts/train_all_rf.py`. The work aligns with SDG 8, SDG 9, and SDG 12, and supports a freemium tourist offering and B2B white-label planning for destination management companies.

Keywords: Intelligent Itinerary, Random Forest, Natural Language Travel Intent, Sri Lanka Tourism, TF-IDF, Personalised Recommendation, Visit-Time Scheduling, Multi-Day Trip Planning

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project supervisor and the academic staff of the Department of Information Technology, SLIIT, for their guidance throughout this research proposal. I acknowledge the Sri Lanka Tourism Development Authority and the published research on machine learning in tourism that informed the methodology of this component.

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the ROC Curve
BB / HB / FB	Bed & Breakfast / Half Board / Full Board
DMC	Destination Management Company
DSR	Design Science Research
LKR	Sri Lankan Rupee
ML	Machine Learning
NL	Natural Language
OOF	Out-of-Fold
PII	Personally Identifiable Information
POI	Point of Interest
PPD	Places Per Day
REST	Representational State Transfer
RF	Random Forest
SDG	Sustainable Development Goal
SLTDA	Sri Lanka Tourism Development Authority
TF-IDF	Term Frequency–Inverse Document Frequency
UAT	User Acceptance Testing
WBS	Work Breakdown Structure

LIST OF APPENDICES

Appendix A: Sample rows from Train_Attractions and Train_Hotels

Appendix B: Sample JSON output of build_itinerary_payload

Appendix C: Web interface screenshots (Attractions → Accommodation → Itinerary)

Appendix D: Console log of the latest successful train_models.py run

Appendix E: Ethical considerations (synthetic training messages)

Appendix F: User acceptance testing survey instrument (optional)

1 INTRODUCTION

1.1 Background and Context

The tourism sector of Sri Lanka is a major source of foreign exchange, with 2.36 million international arrivals in 2025 and a target of 3 million by 2026 [1]. A growing share of these visitors are independent travellers who plan their own multi-day routes. Today that planning is manual: travellers combine blogs, generic maps, and static hotel lists, and must work out for themselves which sites match their interests, how many nights to spend in each city, when each site can be visited, and which hotel fits their budget and meal plan. Poor sequencing and timing contribute to rushed transfers, overlapping visits, and inefficient spending; average daily tourist spending remains approximately USD 148, well below the regional benchmark of USD 300 [1].

The underlying inventory already exists in national-style Excel form: attractions with categories, moods, activities, and official visit start/end times, and registered hotels with BB/HB/FB price bands. What is missing is a single pipeline that takes the tourist's natural-language goal and carries it through every decision in order: NL text → attraction relevance → which cities and how many nights → which visit slot for each attraction → which hotel in each city at the chosen meal-plan price band.

The AI Itinerary Generator implements exactly this sequence as a hybrid of machine learning and deterministic rules. Random Forest models make the preference-driven decisions (need detection, relevance, destination choice, nights, slot choice, places per day, hotel choice), while explicit rules enforce feasibility (travel time, day windows, gaps between visits, and arrival/departure day patterns).

Analysis of Existing Systems and Their Limitations

Table 1 compares currently available platforms against four requirements of Sri Lankan itinerary planning: NL matching against a structured SL catalogue, choice among multiple visit slots per attraction, BB/HB/FB min–max pricing over the full hotel list, and an enforced arrival/departure day pattern.

Table 1: Comparison of Existing Systems and Their Limitations

Platform	Core Capability	Critical Limitation for Sri Lankan Itinerary Planning
Google Travel / Maps	POI search, directions	No NL matching against a structured SL catalogue; no choice among multiple visit slots per attraction; no meal-plan hotel pricing; no arrival/departure day pattern

Platform	Core Capability	Critical Limitation for Sri Lankan Itinerary Planning
TripAdvisor	Community reviews and ratings	Static listings; no day-level schedule; no BB/HB/FB min–max bands across the full hotel registry
Generic AI trip planners	Chat-style suggestions	Not bound to an authoritative SL catalogue (may hallucinate venues); ignore official visit windows; no enforced calendar rules
Sri Lanka Tourism App (SLTDA)	Site descriptions, opening hours, directions	No NL personalisation; no multi-day plan or slot selection; no hotel ranking over ~900 registered properties
Booking.com / Expedia	Hotel and flight booking	Logistics only; no attraction sequencing, visit-slot handling, or arrival/departure day logic
Manual spreadsheets	Full user control	Not scalable; no NL understanding; overlaps and timing must be checked by hand

None of these platforms meets all four requirements in one pipeline. These gaps form the focus of this research.

Proposed Solution

The proposed web module accepts the tourist's trip input, runs it through the offline-trained Random Forest bundle, and returns attraction suggestions, ranked hotels, and a timed multi-day itinerary through a three-step React wizard. Table 2 maps each step of the solution to the implemented modules.

Table 2: Mapping of the Proposed Solution to Implementation Modules

Step	Implementation
User input	Stored in PostgreSQL: `experience_text`, airport arrival/departure dates and times, `budget_split` (accommodation USD converted to LKR per night), `comfort_level`, and `room_type` (mapped to BB/HB/FB)
Suggest places	`suggest_from_trip()` → `run_generate()` → attraction probabilities from `att_model`
Pick hotels (UI)	`rf_hotels_for_destination()` — every hotel in the catalogue receives a `hotel_model` probability; the top-ranked hotel is proposed
Full itinerary	`build_itinerary_payload()` → the same `generate()` call, then database enrichment (contacts, calendar dates, presentation)
Database sync	`seed_data.py` loads both Excel catalogues into PostgreSQL

SDG Alignment

1. **SDG 8 – Decent Work and Economic Growth:** Better-planned trips improve tourist satisfaction and spending efficiency, supporting sustainable employment in tourism services.

2. **SDG 9 – Industry, Innovation and Infrastructure:** The module provides digital tourism infrastructure — a catalogue-grounded ML itinerary engine — that does not currently exist as an integrated national tool.
3. **SDG 12 – Responsible Consumption and Production:** Data-driven routing and realistic timing support informed choices and reduce ad hoc, wasteful travel between sites.

Research Cluster Alignment

This project is aligned with the AI Centre of Excellence (CEAI) at SLIIT, under the research cluster of Machine Learning and Data Science for Real-World Decision Making.

1.2 Literature Review

Ten sources were reviewed, covering tourism recommender systems, Random Forest applications on tabular tourism data, visit-time scheduling, overtourism and sustainable routing, and the Sri Lankan tourism context. Table 3 summarises them.

Table 3: Summary of Literature Review

#	Reference	Method	Key Finding	Gap Identified
1	Bravo et al. (2023) [2]	LR, KNN, DT, RF	ML viable for heritage visitor prediction using public data	Peru-specific; no NL itinerary; no hotel BB/HB/FB integration
2	Kim et al. (2025) [3]	RF, GB, LightGBM	Multi-source features predict trail visitors; temporal flags important	Trail context; no site-level timed slots; no full-registry hotels
3	Mihalić (2020) [4]	Sustainability framework	Overtourism needs demand-side management tools	Theoretical; no deployable itinerary artefact
4	Santos-Rojo et al. (2023) [5]	Bibliometric analysis	Applied technical implementations under-represented	No developing-country catalogue-grounded planner
5	Atrio Lema et al. (2025) [6]	SARIMA, time-series	Site-specific flow prediction feasible with seasonal data	No ML ensemble for POI relevance + hotels jointly
6	Nurhasanah et al. (2022) [7]	SARIMA	National arrival seasonality modelled effectively	Country-level only; no multi-day itinerary output
7	Breiman (2001) [8]	Random Forest	Handles nonlinear mixed features via bagging	Foundational; tourism itinerary application required
8	SLTDA (2024) [1]	Official statistics	2.36M arrivals; 500+ sites; spend gap documented	No ML itinerary engine on registered catalogues
9	Bollenbach et al. (2025) [9]	RF, route graphs	RF supports sustainable tourism routing	Route focus; no visit-window overlap scheduling
10	He et al. (2022) [10]	CNN-GRU	Deep learning for citywide crowds needs large datasets	Not applicable to moderate SL catalogue labels

Discussion of Existing Research Approaches

Bravo et al. [2] apply supervised ML — including Random Forest — to predict visitor-related outcomes at heritage attractions using public official data, confirming that standard supervised ML is feasible for tabular tourism problems. However, the study does not map natural-language goals to catalogue rows, schedule timed visits without overlap, or rank hotels on meal-specific price bands.

Kim et al. [3] show that Random Forest and gradient-based ensembles predict visitor demand using heterogeneous temporal and environmental features, supporting the use of trip dates, comfort, and budget as context features in this component. The study is limited to trail contexts without accommodation scoring.

Mihalić [4] and Santos-Rojo et al. [5] establish that fragmented planning and overtourism call for demand-side digital tools rather than static listings, which motivates a planner that distributes visits across time slots and cities.

Breiman [8] and Bollenbach et al. [9] confirm Random Forest as an interpretable choice for tourism ML where dataset size is moderate, while He et al. [10] show that deep learning is advantageous only with large, fine-grained datasets — which do not exist for Sri Lankan attraction–hotel–time labels.

Relation to the Proposed Design

Building on this literature, the component combines TF-IDF text features with Random Forests. Attraction relevance uses TF-IDF and cosine similarity between the user message and five catalogue fields — `attraction_name`, `category`, `moods`, `activities`, and `details` (`common.attraction_features`). A multi-output Random Forest predicts the user's need over 10 moods and 6 categories (`MOODS` and `CATS` in `common.py`). The literature supports Random Forest at moderate label sizes; in this work the labels are largely synthetic trips generated from catalogue text (`build_training_data._message_from_row`), which keeps the training data reproducible and free of personal data.

1.3 Research Gap

The review highlights five gaps, each addressed directly by the implementation.

Gap 1 — NL-to-multi-field POI matching: Existing tools rely on single keyword lists. This component combines `sim_*` field similarities, need (mood/category) probabilities, and rating as inputs to `att_model`.

Gap 2 — No single deployable model bundle: Recommendation, routing, and lodging are usually handled by separate tools. Here, one `joblib` file holds seven or more trained models plus vectorisers and learned timing parameters (Table 5).

Gap 3 — Visit-time choice: Each attraction has multiple candidate rows (mood × start/end time). `slot_model` chooses the slot, and a runtime greedy scheduler enforces the gap between visits (`timing_params.gap_between_visits_min`) and the learned visiting-hours window (`day_window`).

Gap 4 — Full hotel registry with meal-specific prices: During training, hotel labels are limited to roughly the 25 nearest hotels per destination (`Train_Hotels`), but at runtime every hotel in the registry is scored using `hotel_prices_for_meal(meal)` for the selected BB/HB/FB plan.

Gap 5 — Reproducibility: Examiners can regenerate all results from the same Excel files using `RF_training_data.xlsx`, `build_training_data.py`, `train_models.py`, and `RETRAIN_ALL_RF.bat`.

2 OBJECTIVES

2.1 Main Objective

To design, implement, and evaluate the AI Itinerary Generator: a module that takes natural-language travel intent, airport dates and times, budget, comfort level, and meal plan, and produces a Random Forest-driven multi-day itinerary from authoritative Excel catalogues, exposed through a REST API and a React user interface.

2.2 Specific Objectives

Table 4 lists the specific objectives together with the concrete artefacts that demonstrate each one.

Table 4: Specific Objectives and Deliverables

SO	Objective	Deliverable (actual path / artefact)
SO1	Data integration	Maintain `attractions.xlsx` and `accomadation.xlsx`; sync to `rf_itinerary/rf_itinerary/data/`; `seed_data.py` → PostgreSQL
SO2	Training data design	`build_training_data.py` → sheets Trips, Train_Attractions, Train_Hotels, Train_Visit_Times, Train_PPD, Train_Destinations, Train_Itinerary_Samples, Geo
SO3	Model development	`train_models.py` → `rf_bundle.joblib` containing the models in Table 5
SO4	Runtime generator	`generate_itinerary.py`, `rf_planning.py`, `rf_itinerary_service.py`, `places_rfc_service.py`, `accommodation_rfc_service.py`, `ai_service.build_itinerary_payload`
SO5	Calendar compliance	`travel_days = days - 2`; day 1 = arrival only; 1 < day < days = attractions; last day = shopping + departure; hotel check-in on day 1 or on a change of city
SO6	Evaluation	Report metrics printed by training; API functional tests; baselines = keyword matching and the even-split fallback in `trip_plan_service`
SO7	REST services	Endpoints for trip persistence, suggestion, accommodation ranking, and itinerary build; `bundle_ready()` returns HTTP 503 when the bundle is missing

3 METHODOLOGY

3.1 Research Approach

This research follows Design Science Research (DSR): problem identification, artefact design and development (the Random Forest stack and the web module), and evaluation (training metrics and sample itineraries). The primary predictive technique is supervised learning with Random Forest classifiers and regressors on features combining TF-IDF text vectors, trip context, and catalogue metadata.

3.2 Random Forest Model Stack

Table 5 lists the components of `rf_bundle.joblib` as trained by `train_models.py`. Unless stated otherwise, all forests use `n_estimators=200`, `min_samples_leaf=2`, `random_state=42`, and `n_jobs=-1`.

Table 5: Random Forest Model Stack (rf_bundle.joblib)

Model key	Algorithm	Input features (summary)	Label / role
`msg_vec`, `text_vec`	TF-IDF (1–2 grams on messages and catalogue text)	User message; attraction text fields	Vectorisers (no label)
`need_model`	RF classifier, 200 trees	TF-IDF of message	Multi-label: 10 moods + 6 categories (`LABEL_moods`, `LABEL_categories`)
`att_model`	RF classifier, 200 trees, `class_weight=balanced`	`sim_*` field similarities, mood/category probabilities, rating	`LABEL_relevant` (Train_Attractions)
`dest_clf`	RF classifier	`DEST_COLS`: dest_match, rank, attraction_count, km to airport / best city, travel_days	Destination selected
`dest_days`	RF regressor	Same as `dest_clf`	Nights per destination
`dest_order_model`	RF regressor (optional)	Same as `dest_clf`	Visit order
`slot_model`	RF classifier	start_hour, duration, rating, mood probabilities, `overlaps_planned`	`LABEL_chosen` visit slot
`ppd_model`	RF classifier (trained if ≥ 20 rows)	travel_days, dest_count, day_num	2 vs 3 places per day
`hotel_model`	RF classifier	Price min/max/mid vs budget, comfort one-hot, meal one-hot, distance, rating, category	`LABEL_selected` hotel
`timing_params`	Median from samples (not RF)	Gap between visits, checkout offset	Derived from training visits / trips
`day_window`	Min/max of chosen slots (not RF)	Start hours of chosen slots	Visiting-hours band used by the scheduler

Algorithm Selection

Pure rule-based planners were kept only as a baseline because they cannot learn from labelled relevance and hotel-fit examples across hundreds of POIs. Deep learning was not selected because it requires large, fine-grained datasets [10]. Large language models were not used as the primary engine because, without catalogue grounding, they can produce venues that do not exist in the examiner-verifiable Excel sources. Random Forest was selected because it handles nonlinear, mixed-type features well, performs reliably on moderate, synthetic label sets, provides interpretable feature importances, and is validated in tourism by [2], [3], [8], and [9]. Because the models are trained from the same Excel files, examiners can re-run training and reproduce the reported metrics.

3.3 Training and Validation

- **Need detection:** 5-fold KFold out-of-fold predictions → micro-F1.
- **Attraction relevance:** GroupKFold (5 folds) grouped by `trip_id` → mean AUC, preventing leakage between rows of the same trip.
- **Hotel selection:** GroupKFold (5 folds) → mean AUC.
- **Baselines:** keyword matching for relevance and the even-split day plan in `trip_plan_service` for destination allocation.

No model training takes place during HTTP requests; the runtime only loads the bundle and performs inference.

3.4 Deterministic Rules and Heuristics

Not every decision is learned. The following parts are implemented as explicit rules and are documented as such:

- **Road distance:** haversine distance × `road_factor` 1.35 at an assumed 40 km/h (`common.CONFIG`).
- **Geography:** the airport is fixed as CMB; destination and local-authority coordinates come from the Geo sheet, with a hash-based fallback where coordinates are missing.
- **Scheduler:** attractions are sorted by `att_model` probability; for each, the slot with the highest `slot_model` probability is taken if it lies within the day window and does not clash with an already planned visit (`clash()` plus `gap` in `generate_itinerary`).
- **Day mapping:** `build_day_dest_map` assigns day 1 to the first city, middle days according to `days_int`, and the last day to the last city.

- **Budget conversion:** nightly LKR budget = `budget_split.accommodation` (USD) ÷ trip days × `LKR_TO_USD_RATE` (configurable, default 300).
- **Fallback:** if the RF day plan fails, `trip_plan_service.get_day_plan` uses an even split across destinations; this also serves as the evaluation baseline.

3.5 Tools and Technologies

Table 6: Technology Selection and Justification

Component	Selected Technology	Justification	Alternatives Rejected
ML development	Python 3, scikit-learn, pandas, joblib	Industry standard for RF + TF-IDF; native integration with Flask	R (weaker web glue); MATLAB (licensing)
Excel I/O	openpyxl	Reads catalogues and writes the 8-sheet training workbook	Manual CSV export (loses sheets)
Catalogues	`attractions.xlsx`, `accomadation.xlsx`	Curated, authoritative SL POI and hotel data	Scraped OTAs (legal/quality risk)
Training artefact	`RF_training_data.xlsx`	Reproducible, examiner-readable labels	Ad hoc scripts only
Backend	Flask (Python)	Lightweight REST API in the same language as the ML stack	Django (overhead)
Frontend	React.js	Multi-step wizard: trip input, attractions, hotels, itinerary	Static HTML (unmaintainable)
Database	PostgreSQL	Relational storage for catalogues, trips, and selections	SQLite (concurrency limits)
Interface	REST JSON	Simple contract between React and Flask	GraphQL (unnecessary complexity)

3.6 Data Collection and Ethical Considerations

Catalogues are curated from registered and publicly described tourism inventory. The `user_message` values in the training data are synthetic, generated from catalogue text, so the training sheets contain no tourist PII. Live `experience_text` entered by users is stored by the application according to user consent and processed server-side only. Hotel analysis uses registry fields and meal price bands, not customer transactions. Limitations such as synthetic labels and hash-based fallback coordinates are documented in the thesis and in Appendix E.

4 HIGH-LEVEL ARCHITECTURE

4.1 Three-Layer Architecture

The module follows a three-layer architecture (Figure 1). Attraction suggestion, hotel ranking, and itinerary generation run on the server and are delivered to React pages through REST endpoints.

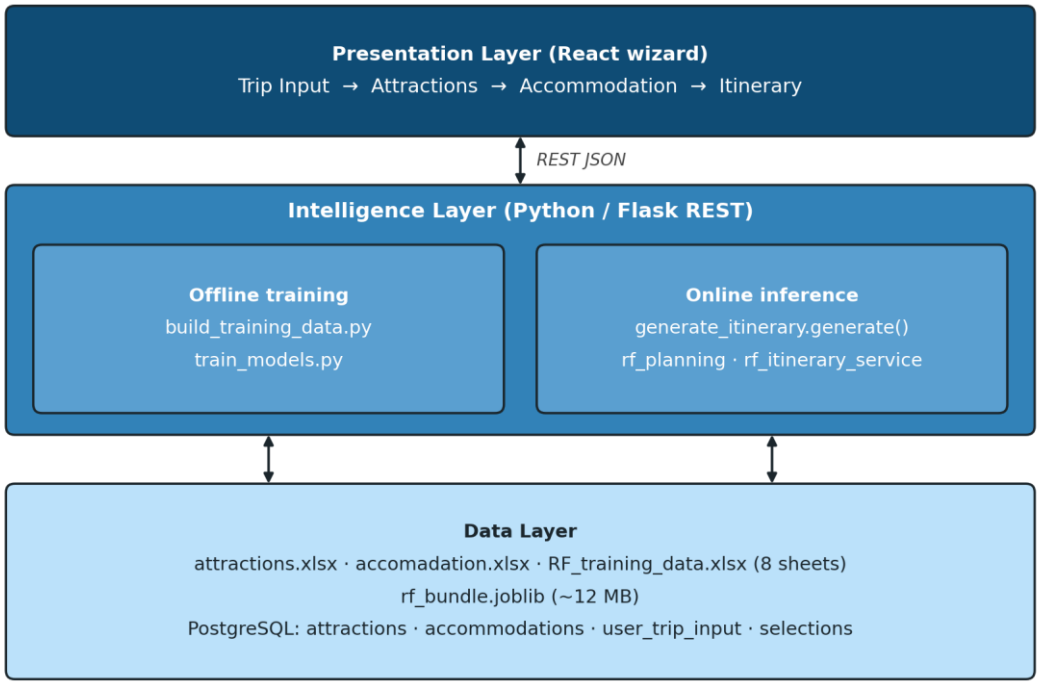


Figure 1: Three-Layer System Architecture

Data layer. Source Excel catalogues in the itinerary `data/` folder and their copies in the RF package (`rf_itinerary/rf_itinerary/data/`); `RF_training_data.xlsx`; the trained `rf_bundle.joblib` (about 12 MB after a typical training run); and PostgreSQL tables for attractions, accommodations, `user_trip_input`, and finalised selections.

Intelligence layer. Offline: `build_training_data.py` and `train_models.py`. Online: `generate_itinerary.generate()`, `rf_planning.allocate_destinations`, and `hotels_for_destination`. The backend glue is `rf_itinerary_service.py`; the bundle is loaded once and cached with `@lru_cache` in the generate module.

Presentation layer. A React wizard: trip input → Attractions (suggestions) → Accommodation (ranked hotels with meal-plan min–max totals) → Itinerary (timeline and print-oriented payload from `itinerary_present_service`).

4.2 Data Flow

1. The user saves a trip: `experience_text`, airport dates and times, budget, comfort, and `room_type`.
2. **Suggest:** `suggest_from_trip` → `understand_need` → `att_model` returns ranked attraction names and a `plan_by_day` skeleton.
3. The user finalises the selected attraction IDs, which are stored in PostgreSQL.
4. **Hotels:** for each destination cluster, `hotel_model.predict_proba` is applied to the full `load_hotels()` frame; the UI shows min–max prices for the selected meal plan.
5. **Build itinerary:** `build_itinerary_payload` → `generate()` → merge with the selected accommodations from the database → enrich with phone numbers, calendar dates, and an arrival narrative.
6. The JSON response is rendered on the itinerary page and is ready for printing.

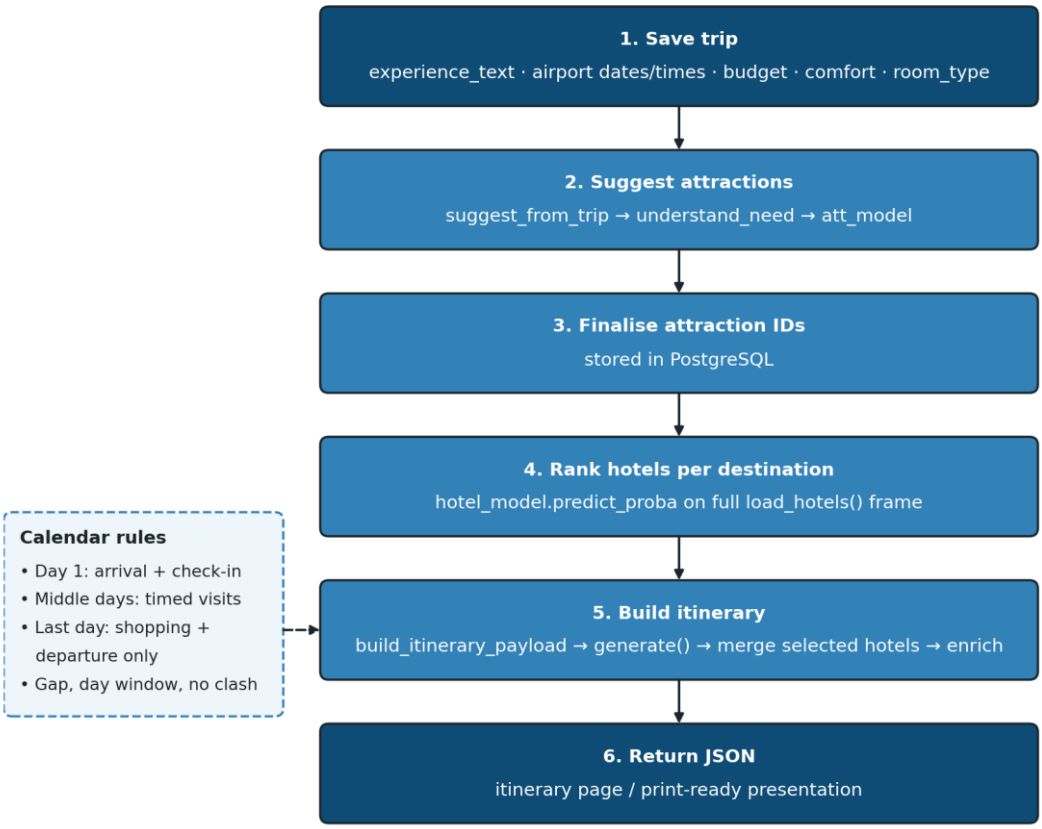


Figure 2: End-to-End Data Flow of Itinerary Generation

4.3 Scalability and Reliability

The bundle is loaded once per service process and reused across requests. Generation cost grows as O(attractions × slots × days), which keeps typical trips within web-acceptable latency. If `rf_bundle.joblib` is missing, the API returns HTTP 503 with a message instructing the administrator to run the retraining script. Database credentials remain in server-side environment variables.

5 USER REQUIREMENTS

Stakeholder Identification

International tourists are the primary users: they enter travel goals, select attractions and hotels, and view timed day-by-day plans. Hotels and destination management companies are indirect beneficiaries of better-matched guest routing. Academics and examiners are stakeholders in the reproducible training pipeline.

Table 7: Functional Requirements

FR#	Requirement Description	Implementation	Priority
FR1	Accept experience text, airport arrival/departure datetime, budget, comfort level, and BB/HB/FB meal plan.	Trip input model; `_meal_from_room_type`, `_comfort_from_trip`	High
FR2	Suggest attractions using RF relevance.	`att_model` + TF-IDF similarity features	High
FR3	Rank / score all hotels per destination.	`hotels_for_destination`, accommodation routes	High
FR4	Generate timed visits with no same-day overlap.	`slot_model` + greedy scheduler + `gap`	High
FR5	Day 1 is arrival only; the last day has no new sightseeing.	`generate_itinerary` loops for `i == 1`, `1 < i < days`, `i == days`	High
FR6	Show meal-specific LKR min–max per night and per stay.	`hotel_prices_for_meal`, AccommodationPage (UI)	High
FR7	Persist trips and selections via REST.	`trip_input.py`, places / accommodation routes	High
FR8	Provide a rich, print-ready itinerary presentation.	`itinerary_present_service`, `enrich_itinerary_presentation`	Medium
FR9	Allow retraining from updated Excel files.	`RETRAIN_ALL_RF.bat`, `train_all_rf.py`	Medium
FR10	Expose an ML status endpoint.	`places_planner` bundle check	Medium

Table 8: Non-Functional Requirements

NFR#	Category	Measurable Requirement
NFR1	Performance	Suggest and generate shall complete within 30 seconds for typical multi-city trips under normal load.
NFR2	Accuracy	Attraction and hotel ranking AUC and need micro-F1 shall be reported from the training logs and shall exceed keyword baselines.
NFR3	Availability	A missing `rf_bundle.joblib` shall return an explicit HTTP 503 with retrain instructions.
NFR4	Usability	The wizard shall function on desktop and mobile browsers.
NFR5	Security	No secrets (API keys, connection strings) shall appear in frontend code.
NFR6	Privacy	Training artefacts shall contain no tourist PII; trip text is handled per consent.
NFR7	Maintainability	The RF package path, training scripts, and bundle version shall be documented; catalogue updates require only `RETRAIN_ALL_RF.bat`, not code changes.

6 COMMERCIALIZATION PLANS

Target Market

The primary B2C segment is international tourists visiting Sri Lanka — approximately 2.36 million arrivals in 2025, with a target near 3 million by 2026 [1] — especially independent travellers and families who self-plan multi-city routes. The secondary B2B segment is boutique destination management companies (DMCs) and registered hotels seeking a white-label planner grounded in Sri Lankan inventory.

Revenue Model

- **B2C freemium:** basic itinerary generation is free; premium features include branded export, unlimited revisions, and multi-trip storage.
- **B2B SaaS:** a white-label planner for DMCs using the same Excel catalogues and retraining pipeline, so each DMC can maintain its own inventory.

Competitive Advantage

This is the only Sri Lanka-focused system described that combines natural-language intent, official visit windows, and Random Forest ranking of the full BB/HB/FB hotel registry in a single deployable bundle — capabilities absent from generic global trip planners.

Projected Revenue (Conceptual Estimate)

If the platform reaches 4,000 monthly active users with 8% premium conversion at LKR 600 per trip, B2C revenue is about LKR 192,000 per month. Twenty B2B seats at LKR 8,000 per month add LKR 160,000, for a total of roughly LKR 352,000 per month — enough to cover hosting and maintenance after launch.

Intellectual Property

Intellectual assets include the trained `rf_bundle.joblib`, the `RF_training_data.xlsx` labelling methodology, and the documented REST API. Catalogue compilations remain attributable to their curators and SLTDA-published descriptors.

7 GANTT CHART

Figure 3 shows the development timeline from March to October 2026: catalogue QA, building `RF_training_data.xlsx`, training and metrics, Flask services, React pages, evaluation, and the thesis.

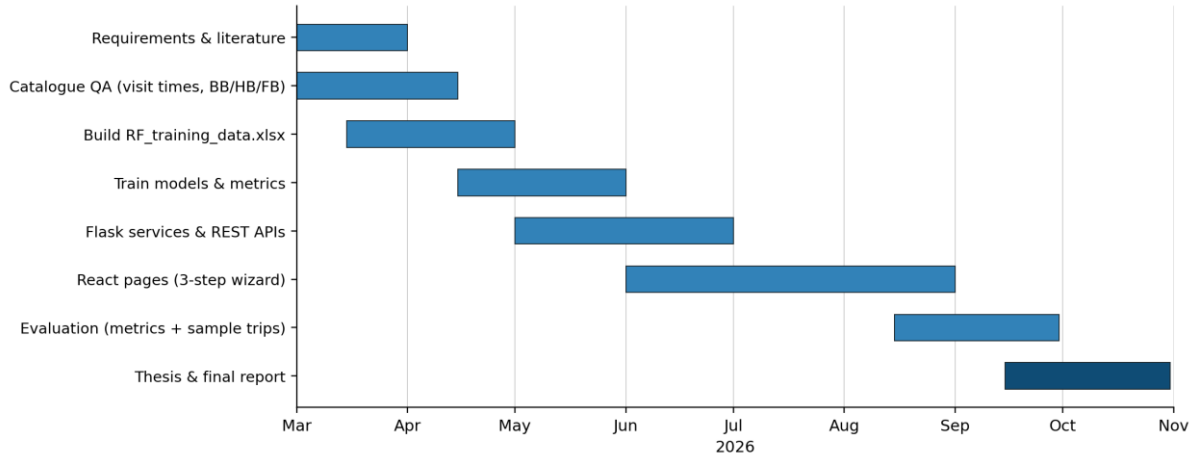


Figure 3: Gantt Chart — March to October 2026 Development Timeline

8 SYSTEM DIAGRAM

Figure 4 shows the training and deployment pipeline as implemented in the codebase.

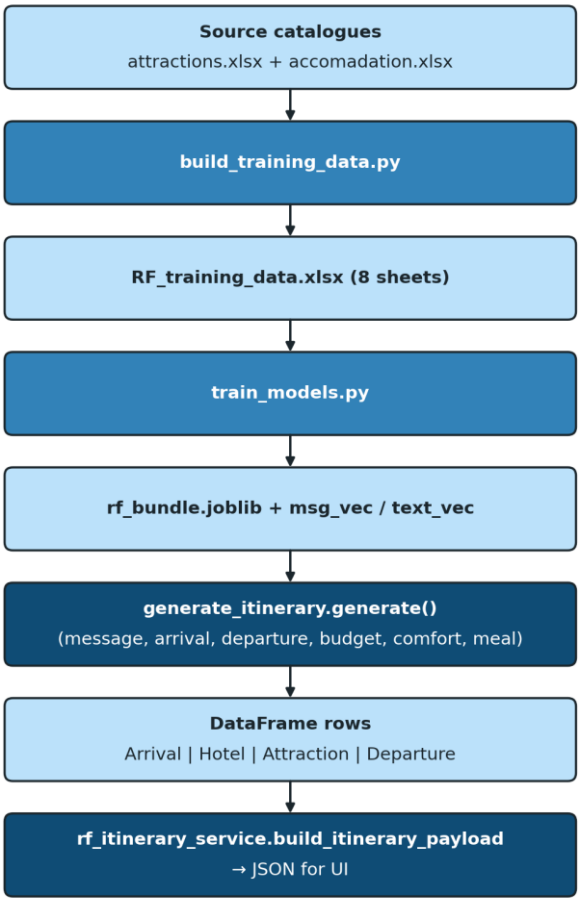


Figure 4: Random Forest Training and Deployment Pipeline

Runtime Inputs and Inference Sequence

At runtime, `generate()` receives: `message` = `experience_text` (or the joined experiences); `arrival` / `departure` = airport date and time strings; `budget_lkr` = accommodation USD per night converted to LKR; `comfort` = Budget | Standard | Comfortable | Luxury; and `meal` = BB | HB | FB derived from `room_type` (for example, `double_hb` → HB). Inside `generate()`, the models run in sequence: mood/category need → attraction probabilities → destinations, nights, and order → places per day → visit slots → hotel for each city on the route → a row-based itinerary table (Arrival | Hotel | Attraction | Departure).

9 CONCLUSION

This proposal presents the AI Itinerary Generator, a web module that converts a tourist's natural-language goals, airport dates and times, budget, comfort level, and meal plan into a feasible multi-day itinerary for Sri Lanka. It addresses five gaps: NL-to-multi-field POI matching, the lack of a single deployable model bundle, learned visit-time choice, full-registry hotel scoring with meal-specific prices, and reproducibility.

The solution combines a multi-model Random Forest stack with TF-IDF features, grounded in authoritative Excel catalogues, with explicit calendar rules: arrival and check-in on day one, timed visits on middle days, and shopping and departure on the last day. Evaluation relies on the micro-F1 and AUC metrics produced by `train_models.py`, and the entire pipeline can be regenerated with `RETRAIN_ALL_RF.bat`. The work is technically feasible within the March–October 2026 timeline, aligns with SDG 8, 9, and 12, and offers a credible commercial path through freemium and B2B offerings.

10 BUDGET AND BUDGET JUSTIFICATION

Table 9: Budget Breakdown and Justification

Item	Description	Cost (LKR)	Justification
Cloud hosting	Flask backend + React frontend + PostgreSQL (3-month demo)	11,500	Required for testing beyond localhost and for the final demonstration
Storage / backup	Storage for `rf_bundle.joblib` (~12 MB) and database snapshots	5,000	Versioned model artefacts and DB backups
Development tools	Open-source Python stack; IDE / contingency	3,000	Zero ML library licensing; tooling buffer
Field validation	Travel to validate 3–5 sample routes (compare actual times vs catalogue visit windows)	12,000	Qualitative validation of timings and routing
Documentation	Printing, binding, contingency	6,000	Submission and unexpected costs
Total		**37,500**	

Note: All ML and frontend libraries are open source. No paid third-party POI API is required because the catalogues are curated locally.

11 WORK BREAKDOWN STRUCTURE

Table 10: Work Breakdown Structure

Task ID	Task	Deliverable	When
T1	Validate Excel columns (visit times, BB/HB/FB)	Updated catalogues	March–April 2026
T2	Run `build_training_data.py`	`RF_training_data.xlsx`	March–April 2026
T3	Run `train_models.py`	`rf_bundle.joblib` + console AUC/F1	April–May 2026
T4	Run `seed_data.py`	Populated PostgreSQL	May 2026
T5	Wire `rf_itinerary_service`	Suggest and itinerary APIs	May–June 2026
T6	React Attractions / Accommodation / Itinerary pages	End-to-end demo	June–August 2026
T7	Evaluation chapter	Screenshots, metric table, 2–3 sample trips	August–September 2026
T8	Thesis	Final report	September–October 2026

12 REFERENCES

[1] Sri Lanka Tourism Development Authority, SLTDA Year-In-Review 2024, Colombo: SLTDA, 2024.

[2] J. Bravo, R. Alarcón, C. Valdivia, and O. Serquén, "Application of machine learning techniques to predict visitors to the tourist attractions of the Moche Route in Peru," Sustainability, vol. 15, no. 11, p. 8967, Jun. 2023. https://doi.org/10.3390/su15118967

[3] G.-H. Kim, H.-J. Joo, and S. Lee, "Visitor number prediction for Daegwallyeong Forest Trail using machine learning," Sustainability, vol. 17, no. 13, p. 6061, Jul. 2025. https://doi.org/10.3390/su17136061

[4] T. Mihalić, "Conceptualising overtourism: A sustainability approach," Annals of Tourism Research, vol. 84, p. 103025, Sep. 2020. https://doi.org/10.1016/j.annals.2020.103025

[5] C. Santos-Rojo, M. Llopis-Amorós, and J. M. García-García, "Overtourism and sustainability: A bibliometric study (2018–2021)," Technological Forecasting and Social Change, vol. 188, p. 122285, Mar. 2023. https://doi.org/10.1016/j.techfore.2022.122285

[6] Y. Atrio Lema, I. Neira Gómez, and M. L. del Río, "Predictive modelling for sustainable pilgrim flow management on the Camino de Santiago," PASOS, vol. 23, no. 2, pp. 433–448, 2025. https://doi.org/10.25145/j.pasos.2025.23.031

[7] D. Nurhasanah, A. M. Salsabila, and M. D. Kartikasari, "Forecasting international tourist arrivals in Indonesia using SARIMA model," Enthusiastic, vol. 2, no. 1, pp. 19–25, 2022. https://doi.org/10.20885/enthusiastic.vol2.iss1.art3

[8] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001. https://doi.org/10.1023/A:1010933404324

[9] J. Bollenbach, D. Rebholz, and R. Keller, "The road not taken: Representing expert knowledge for route similarities in sustainable tourism using machine learning," Electronic Markets, vol. 35, no. 1, pp. 1–21, 2025. https://doi.org/10.1007/s12525-024-00714-6

[10] Z. He, C.-Y. Chan, J. Zhang, and L. Sun, "Forecasting the crowd: An effective and efficient neural network for citywide crowd information prediction," Transportation Research Part C, vol. 143, p. 103799, 2022. https://doi.org/10.1016/j.trc.2022.103799

13 APPENDICES

The following artefacts will be attached at final submission:

- **Appendix A:** Sample rows from `Train_Attractions` and `Train_Hotels`.
- **Appendix B:** Sample JSON returned by `build_itinerary_payload`.
- **Appendix C:** Screenshots of the three UI pages (Attractions, Accommodation, Itinerary).
- **Appendix D:** Console log of the latest successful `train_models.py` run, including dataset sizes (e.g., 171 trips, ~890 hotels, ~343 attractions in the most recent run) and the printed AUC/F1 metrics.
- **Appendix E:** Ethical considerations, including the generation of synthetic training messages.
- **Appendix F:** User acceptance testing survey instrument (optional).